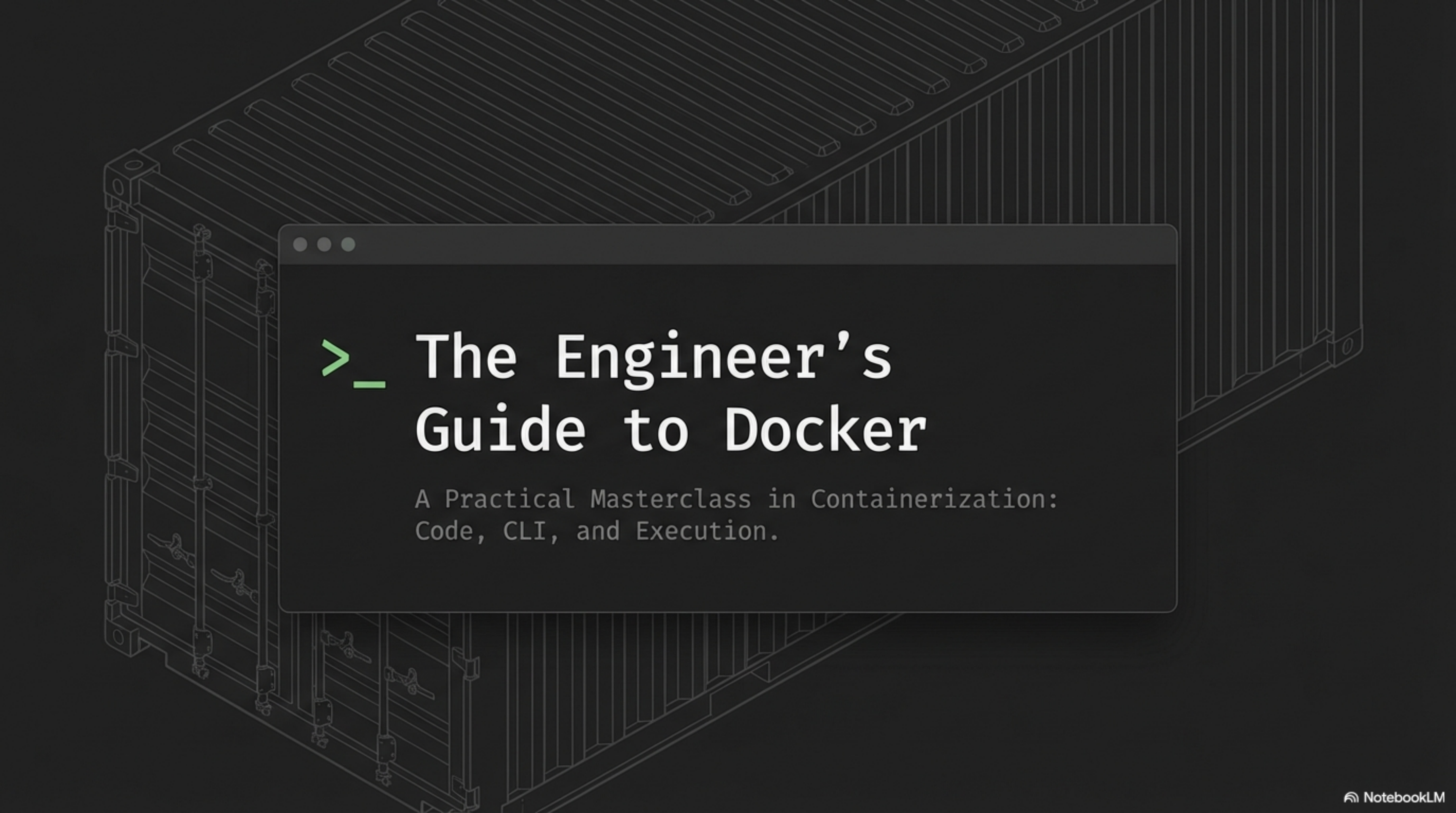
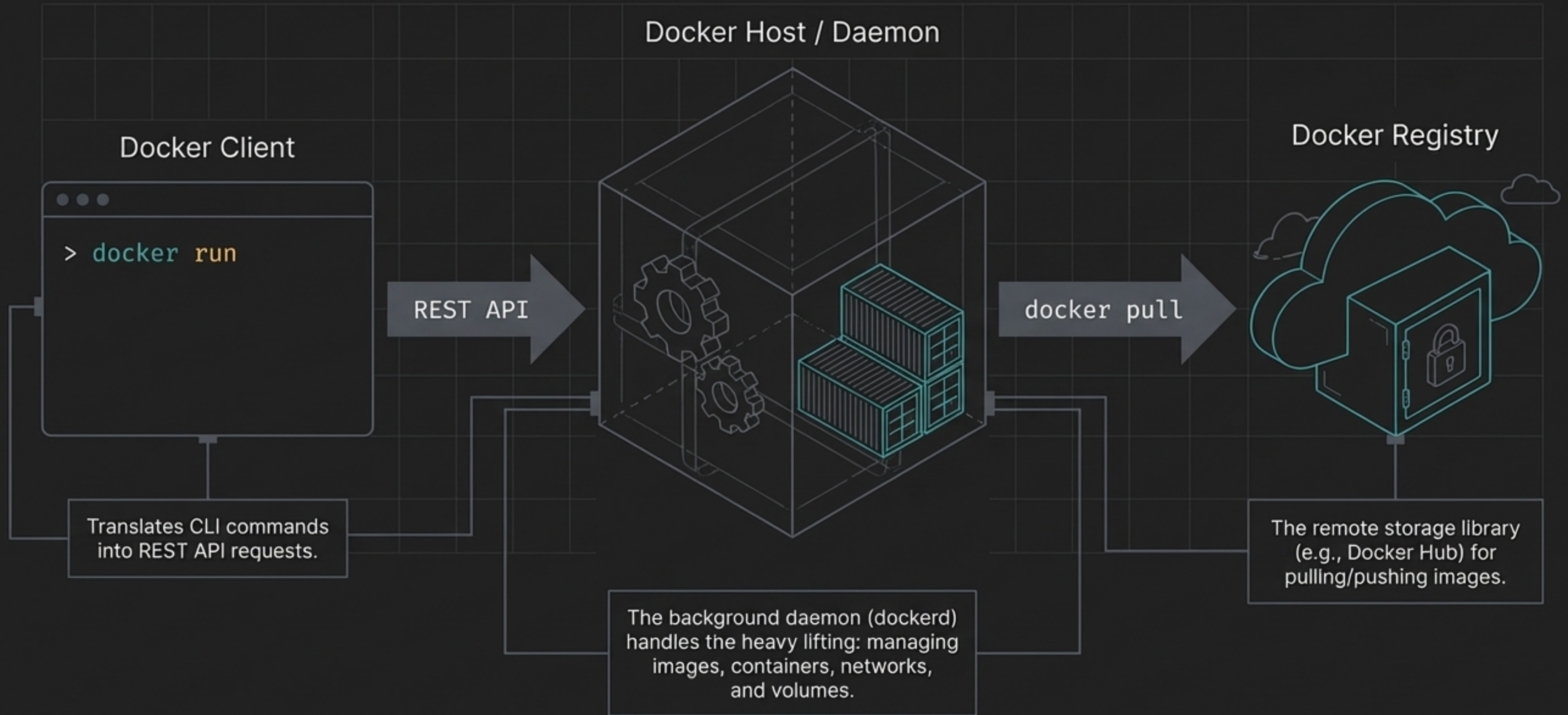




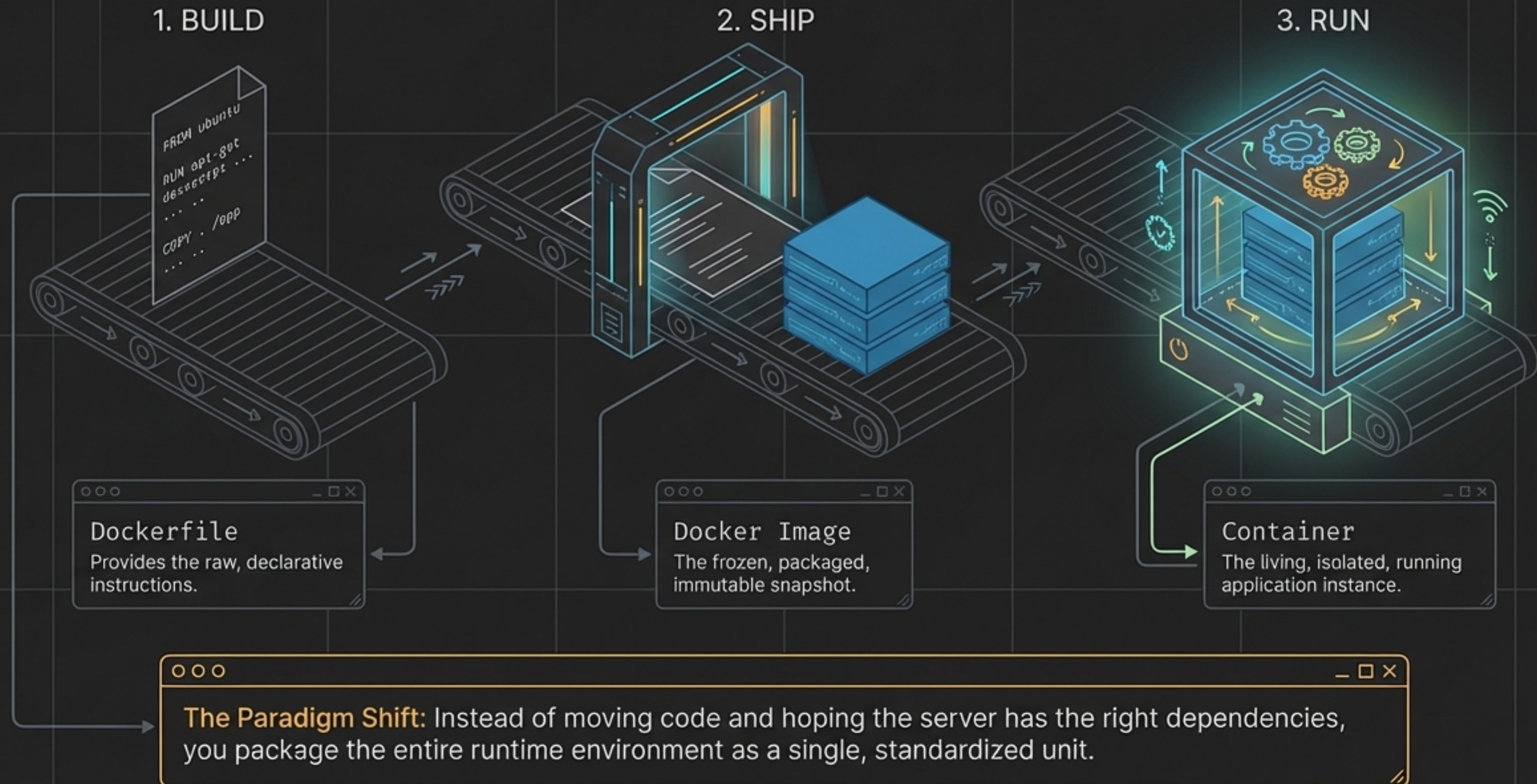
>_ The Engineer's Guide to Docker

A Practical Masterclass in Containerization:
Code, CLI, and Execution.

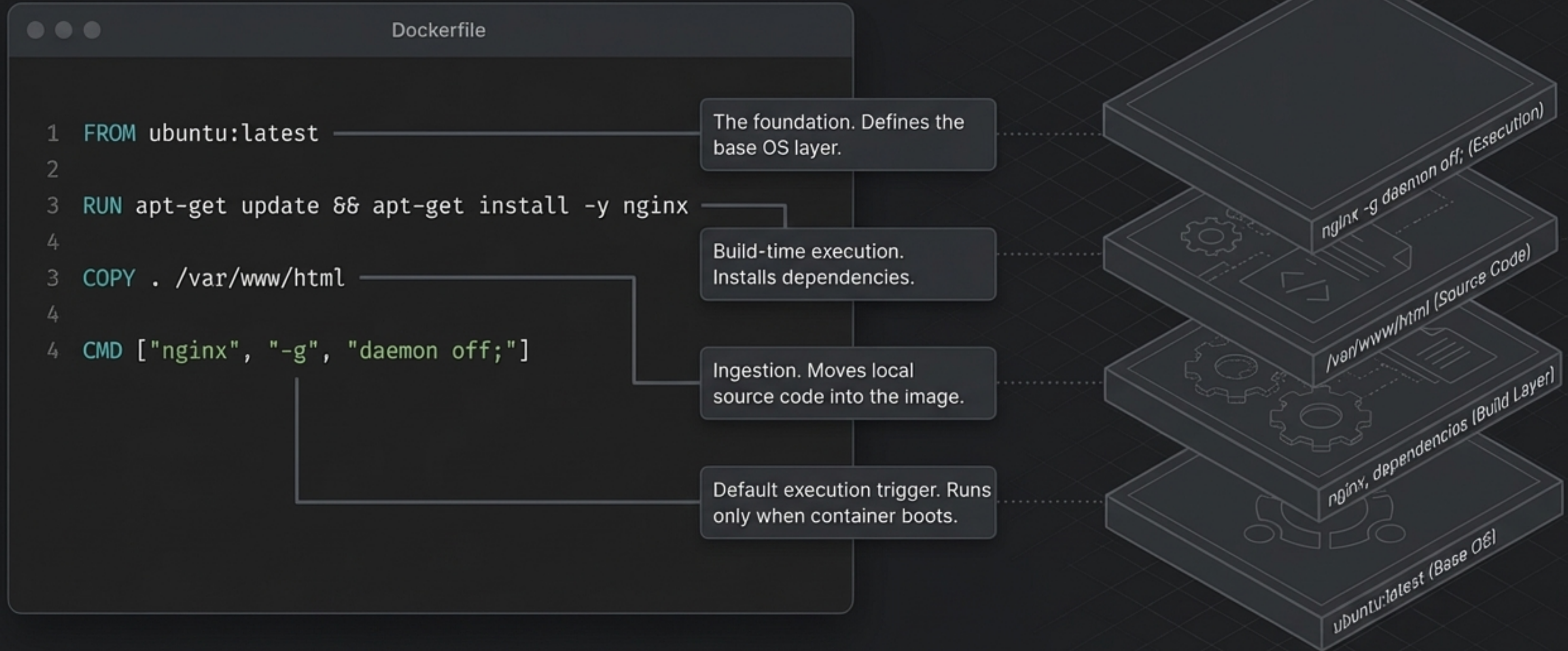
The Underlying Engine



The Assembly Line: Build, Ship, Run



BUILD | The Blueprint



Terminal - Docker Build Process

```
> docker build -t my-app .
```

The Base Trigger

Instructs the Docker Daemon to locate and parse the Dockerfile instructions.

The Tag Flag

Assigns a human-readable repository name (and optional version tag like :v1) to the final compiled image.

The Build Context

Explicitly tells the Daemon to look in the *current working directory* for the Dockerfile and source files.

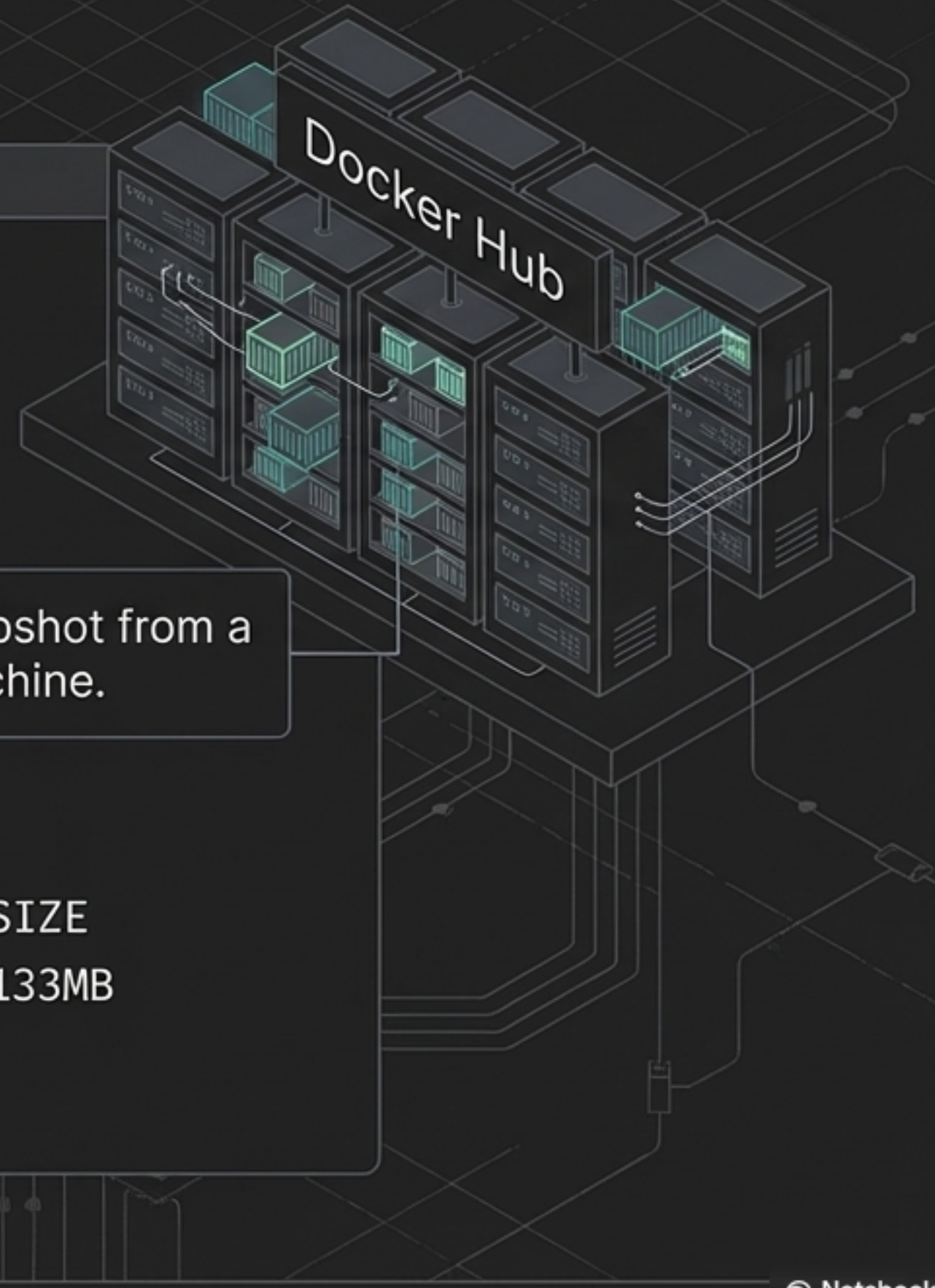
SHIP | The Global Registry

```
> docker pull nginx:latest
latest: Pulling from library/nginx...
Status: Downloaded newer image for nginx:latest
```

Downloads a stateless, pre-built snapshot from a global registry to your local Host machine.

```
> docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
nginx	latest	a24bb4013296	2 days ago	133MB



RUN | Booting the Container

```
> docker run -d -p 8080:80 nginx
```

```
graph TD; A["> docker run -d -p 8080:80 nginx"] -- blue --> B["Execution"]; A -- orange --> C["Detached Flag"]; A -- orange --> D["Port Mapping"]; A -- green --> E["Target Image"];
```

Execution

Creates a new container from a specified image and starts it.

Detached Flag

Forces the container to run in the background. Frees up your terminal prompt instead of locking it to the stdout stream.

Port Mapping

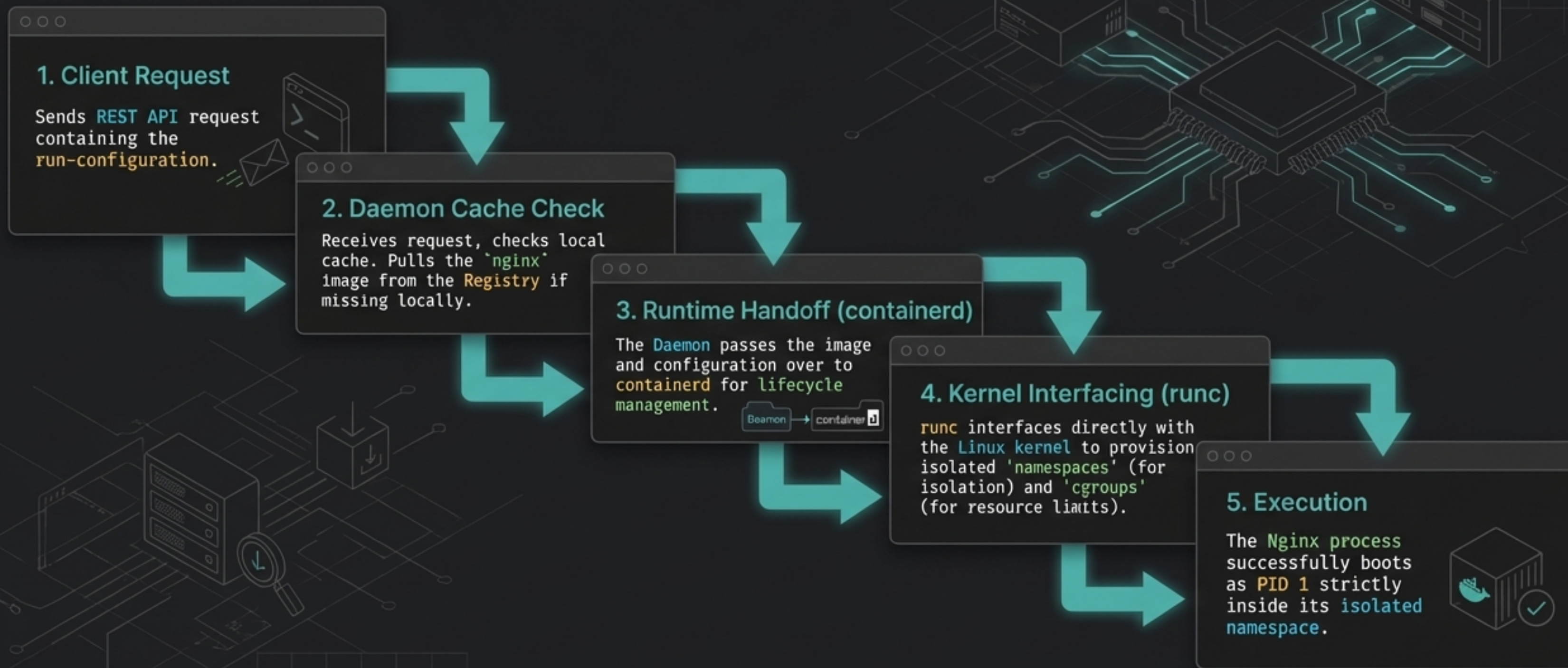
The network bridge. Maps external Port 8080 on the Host machine directly to internal Port 80 inside the isolated container.

Target Image

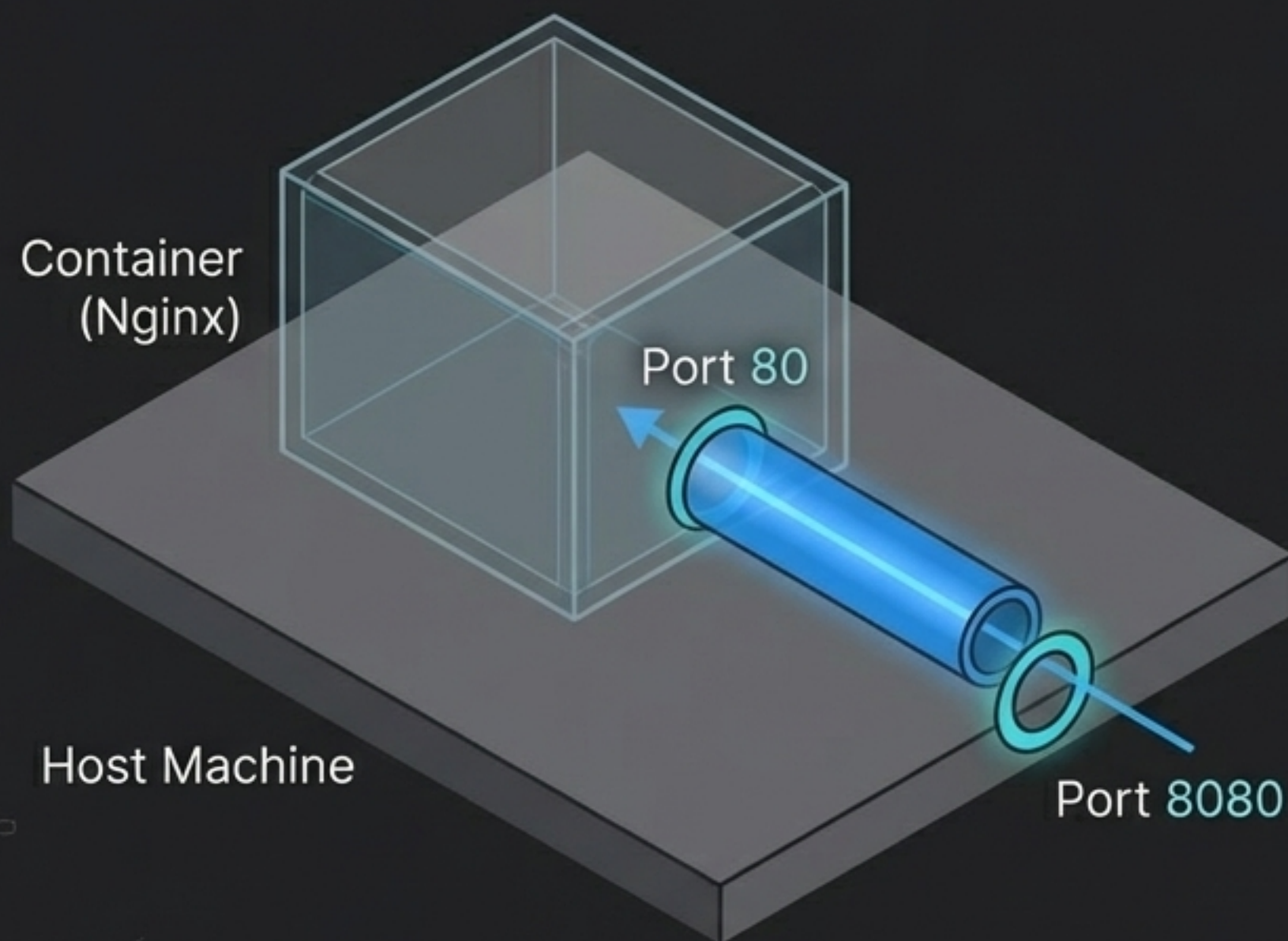
The specific image acting as the blueprint for this new container instance.

Under the Hood of `docker run`

What actually happens when you hit enter?



Navigating Port Isolation



Core Principle: Containers are completely isolated by default. Port mapping is required to intentionally punch a secure, specific routing tunnel for external traffic.

ENGINEER'S CONSOLE | Heartbeat & Status

CLI Evolution

Old: `docker ps` -> New: `docker container ls`

```
> docker container ls -a
```

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS
c4b8e9f2a176	nginx	"nginx -g 'daemon ...'"	Up 2 minutes	0.0.0.0:8080->80/tcp
a9f4c3b2e1d0	redis	"docker-entrypoint..."	Exited (0)	

-a flag: Shows ALL containers, including crashed or stopped instances (crucial for debugging).

```
> docker container top c4b8e9f2a176
```

Acts like the Linux ``top`` command, displaying the live, running processes strictly isolated inside the target container.

ENGINEER'S CONSOLE | The Portal

```
user@macbook ~$ docker exec -it c4b8e9f2a176 bash
root@c4b8e9f2a176:/#
```

Executes a **NEW** command inside an **ALREADY RUNNING** container.

Interactive: Keeps **STDIN open** even if not attached, allowing you to type inputs.

TTY: Allocates a **pseudo-TTY**, formatting the session to look and act like a **native, responsive terminal**.

The **specific process** to launch—opening a **live bash shell session** directly inside the isolated environment.

ENGINEER'S CONSOLE | Lifecycle & Cleanup

> `docker stop [container_id]`

Gracefully sends a SIGTERM to halt the primary process.

> `docker rm -f [container_id]`

Deletes the stopped container footprint. The `-f` flag force-kills it if still running.

> `docker image rm [image_id]`

Deletes the underlying cached image to free up local disk space.

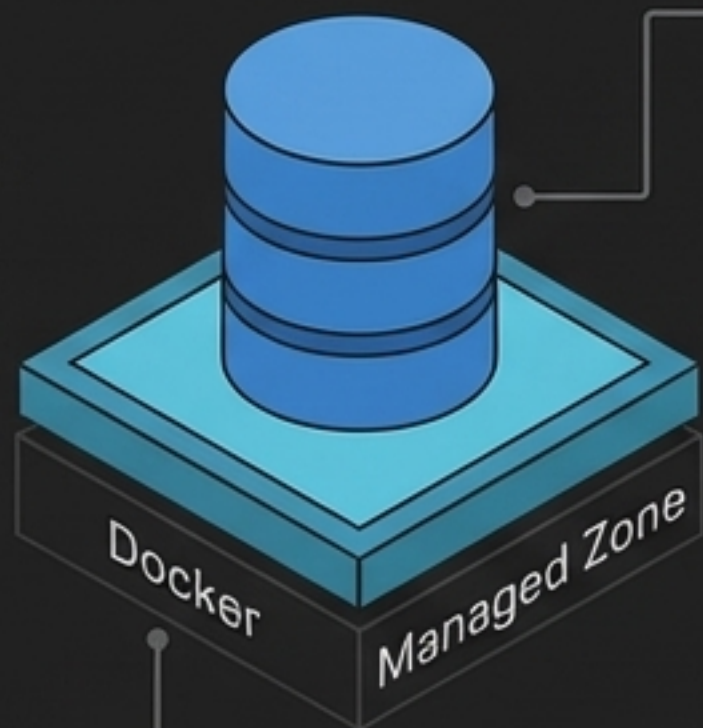
> `docker system prune`

The Nuclear Option: Wipes all stopped containers, unused networks, dangling images, and build cache in a single stroke.

The Persistence Matrix

The Trap: Container filesystems are ephemeral. If a container crashes, internal data is wiped clean.

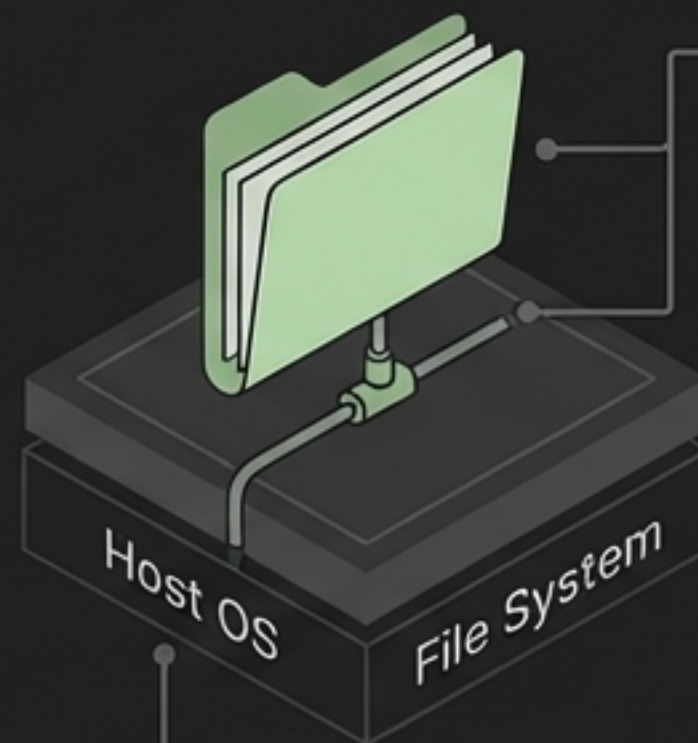
Docker Volumes



Docker Volumes

- **Management:** Fully managed by Docker CLI/API.
- **Location:** Stored in `/var/lib/docker/volumes/`
- **Isolation:** Safely abstracted away from the core Host OS.
- **Use Case:** Best practice for Databases.

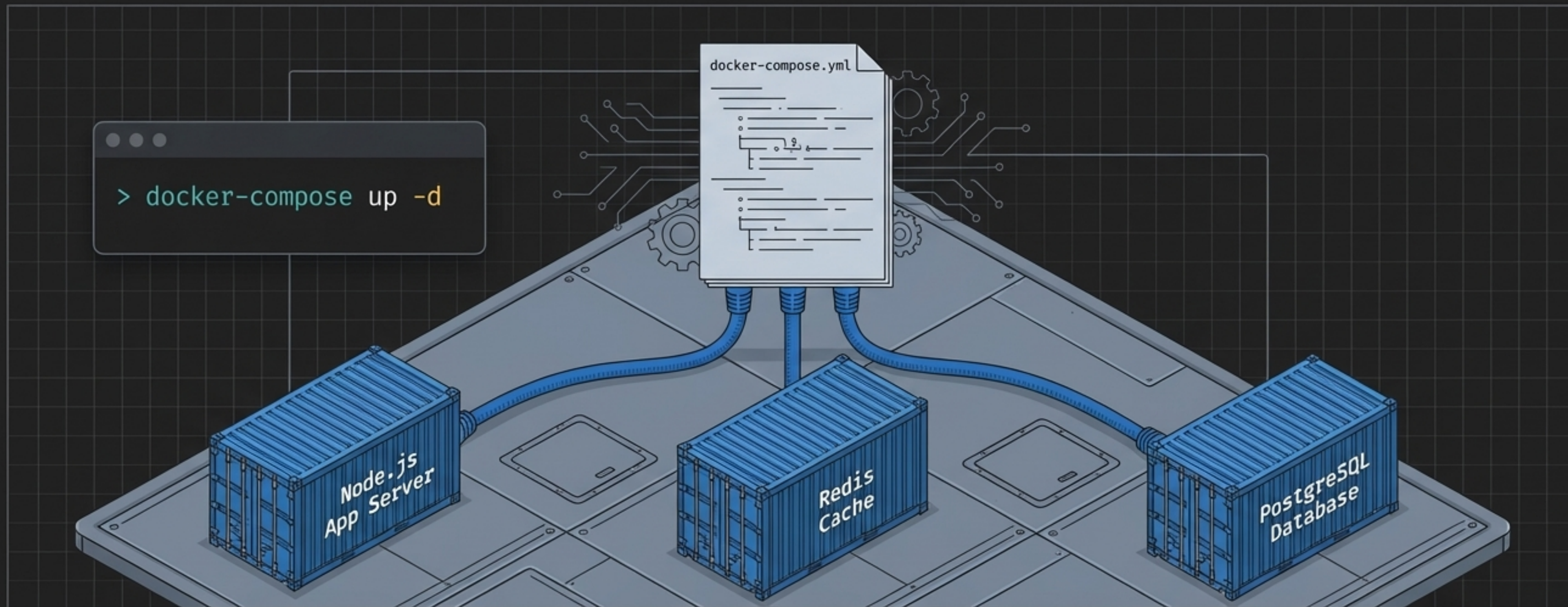
Bind Mounts



Host OS File System

- **Management:** Relies strictly on Host machine directory structure.
- **Location:** Maps a specific Host folder directly.
- **Isolation:** None. Container can alter sensitive host files.
- **Use Case:** Local developer live-reloading of source code.

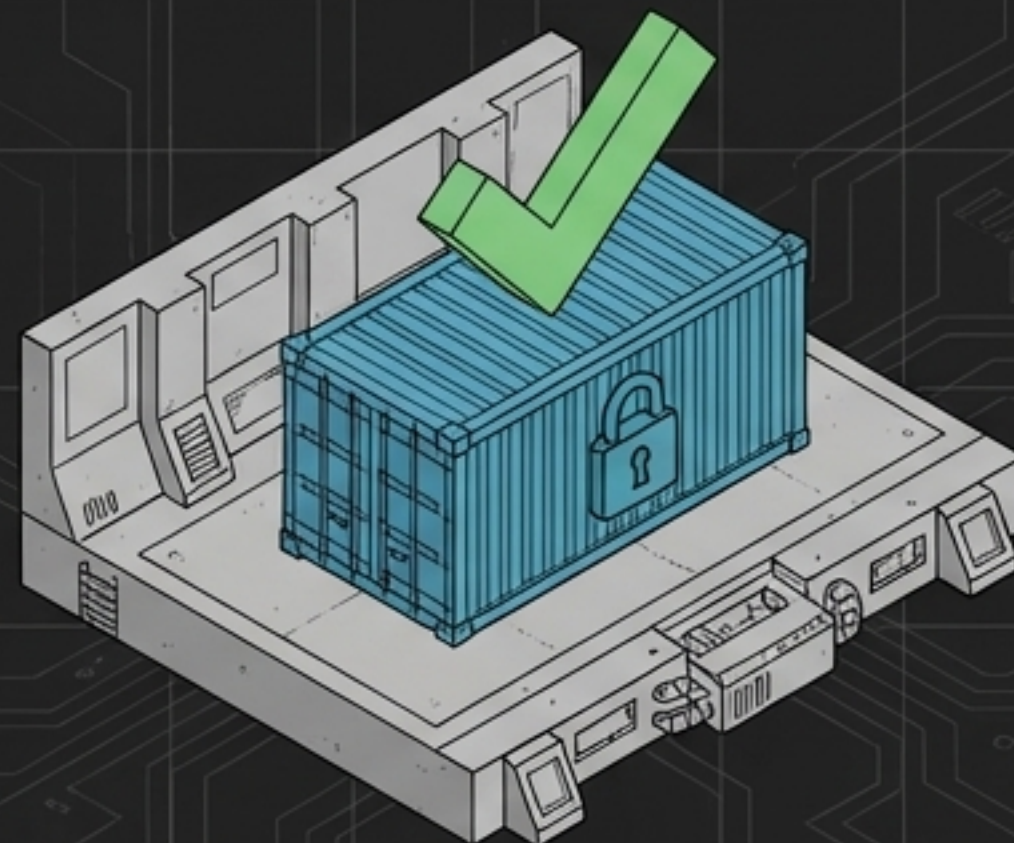
Synthesis | Orchestrating the Environment



Moving beyond the individual prompt: Docker Compose replaces the manual execution of dozens of complex ``docker run`` commands, custom networks, and volume attachments by declaring the entire desired state in a single YAML file.

Synthesis: Docker is not just a local tool; it is the fundamental, standardized currency of the modern CI/CD pipeline. From Git to automated build, to registry, to production.

The Golden Path



Automate, Measure, Learn

Treat infrastructure as code and version everything. Manual server intervention is an anti-pattern.

Shift-Left Security

Embed testing and security vulnerability gates directly into the Dockerfile build pipeline, not as an afterthought.

Scale Responsibly

Master the isolation, networking, and storage of a single local container before attempting to orchestrate a massive fleet with Kubernetes.

The 'works on my machine' era is definitively over. Welcome to the digital shipyard.